Ders 6 Genişletilmiş Notu: Iterator, Memento ve Observer Kalıpları

Ders 6 Genişletilmiş Notu: Iterator, Memento ve Observer Kalıpları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Iterator Kalıbı: Amaç ve Sorun
- 2. Iterator Kalıbı: Birinci ve İkinci Çözüm
- 3. Iterator Gerçekleme Ayrıntıları
- 4. Iterator Uygulama Örneği: Online Yemek Siparişi
- 5. Memento Kalıbı: Amaç, Yapı ve Etkileşim
- 6. Memento'yu Korumak: Dile Göre Çözümler
- 7. Memento Kod Gösterimi ve Kırma Denemeleri
- 8. Memento Uygulama Örneği: Geri Alma ve Yineleme
- 9. Observer Kalıbı: Amaç ve Yapı
- 10. Observer Gerçekleme Ayrıntıları
- 11. Observer Uygulama Örneği: Yeni Müşteri Eklenmesi
- Örnek Sorular
- Dersin Toparlaması

1. Iterator Kalıbı: Amaç ve Sorun

Bir önceki derste davranışsal kalıplara Chain of Responsibility ve Command ile girilmişti. Bu ders davranışsal grubun sıradaki kalıbı Iterator ile başladı. Hoca "iterator" sözcüğünü Türkçeye "gezgin" diye çevirdiğini söyledi.

1.1 Amaç

Slayttaki amaç şudur (ekrandan, 00:06:10): parçalardan oluşan bir bileşenin parçalarını, bileşenlerin iç yapısını açığa çıkarmadan, istenen bir biçimde dolaşmak. Yani bir parça-bütün ilişkisindeki yapının bileşenleri arasında, o bileşenlerin iç yapısına bağımlı olmadan gezinmek istiyoruz. Bunun en basit örneği bir veri yapısı içinde dolaşmaktır.

1.2 Sorun

Sorun, veri yapısının arayüzünü birçok farklı dolaşım türüyle şişirmek istememizdir. Veri yapısının türüne göre çok farklı dolaşım biçimleri vardır. Hoca bunu iki örnekle anlattı:

- İkili ağaçta (binary tree) her düğümün sol ya da sağ çocuğu olabilir. "Önce sol çocuk varsa sola git, çocuk yoksa veriyi al, sonra sağ çocuk varsa sağa git" biçimindeki dolaşım LDR (Left-Data-Right) dolaşımıdır. L, D ve R harfleri üç sırada $3! = 6$ farklı biçimde dizilebildiği için LDR, RDL, DLR, DRL gibi altı dolaşım türü vardır.
- Bağlı listede, bağların tek yönlü ya da çift yönlü olmasına ve listenin dairesel (circular) olup olmamasına göre farklı dolaşımlar söz konusu olur.

Bütün bu dolaşımları veri yapısının kendi arayüzüne metot olarak eklemek arayüzü gereksiz yere büyütür.

1.3 Java'nın Iterator'ı ve KISS İlkesi

Java'da veri yapılarını kullananlar bilir: herhangi bir koleksiyondan kendisini gezmek üzere bir `Iterator` nesnesi istenebilir. Ama bu nesne Iterator tasarım kalıbına tam uymaz; kalıbın biraz daha basitleştirilmiş hâlidir. Hoca bunu küçümsemek için söylemediğini vurguladı: basit demek kötü demek değildir ve çoğu durumda Java'nın `Iterator`'ı işimize yarar.

Burada KISS ilkesi devreye girer: "Keep It Simple, Stupid". Hoca, kısaltmanın tek başına okunca sevimli geldiğini, ama asıl anlamının şu olduğunu söyledi: yapılacak işi olması gerekenden karmaşık yaparsak aptalca bir iş yapmış oluruz. Seçenekler ihtiyaç kadar tutulmalıdır. Bu yüzden Java'nın `Iterator`'ı işimizi görüyorsa onu kullanıp geçeriz. Kalıbın asıl gücü, örneğin Composite'teki gibi parça-bütün hiyerarşilerinde dolaşmak gerektiğinde ortaya çıkar.

2. Iterator Kalıbı: Birinci ve İkinci Çözüm

Slaytlar kalıba adım adım ulaşan çözüm varyasyonlarını tartışır.

2.1 Birinci Çözüm

Slayttaki birinci çözüm (ekrandan, 00:10:00) iki sınıftan oluşur:

```
List                          ListIterator
-----------------             ----------------
Count()          <---list---  First()
Append(Element)               Next()
Remove(Element)               IsDone()
...                           CurrentItem()
                              ----------------
                              index
```

`List` sınıfında eleman sayısını veren `Count()`, eleman ekleyen `Append()`, eleman silen `Remove()` ve listede gereken diğer metotlar bulunur. `ListIterator` ise liste yapısında dolaşmak üzere özelleşmiş bir sınıftır. Dolaşacağı listeyi bir sahiplik ilişkisiyle bilir ve o anda listenin hangi elemanında kaldığını `index` değişkeninde tutar. Dört metodu vardır: ilk elemanı getir, sonraki elemanı getir, dolaşılacak eleman kaldı mı, şu anki elemanı getir. Başka metotlar da düşünülebilir.

- Güçlü yönü: veri yapısı ile dolaşım biçimi birbirinden soyutlanmıştır. Veri yapısı, nasıl dolaşılacağından bağımsız tanımlanabilir.
- Zayıf yönü: bu kez de belli bir veri yapısı ile belli bir dolaşım biçimi arasında bağlılık oluşmuştur. Belli bir yapıyı dolaşmak için belli bir gezgin sınıfına ihtiyaç vardır.

Bu bağlılığı azaltmak için genel bir liste yapısı arayüzü ve genel bir dolaşım biçimi arayüzü oluşturulabilir. Hoca, bu noktada aklına ilk gelenin Bridge kalıbı olduğunu söyledi. Daha önce görülen Bridge örneğinde şekil ile çizimi birbirinden ayrı tutulmuştu; burada da benzer bir yöne gidilebilir.

2.2 İkinci Çözüm

İkinci çözüm (ekrandan, 00:12:10) iki soyut tip üzerine kurulur:

```
                Client
               /      \
              v        v
  AbstractList            Iterator
  ------------------      --------------
  CreateIterator()        First()
  Count()                 Next()
  Append(Item)            IsDone()
```

```
Remove(Item)            CurrentItem()
...
   ^                        ^
   |                        |
List      - - - - - ->  ListIterator     (List, ListIterator'ı oluşturur;
SkipList  - - - - - ->  SkipListIterator  gezgin kendi listesini bilir)
```

Soyut `AbstractList` sınıfının `List` ve `SkipList` gibi alt sınıfları vardır. Skip list, arada bazı elemanların atlanabildiği bir liste olarak düşünülebilir; hoca çift bağlı ya da dairesel liste gibi başka seçeneklerin de olabileceğini söyledi. Her somut liste için ona uygun somut bir gezgin bulunur: `ListIterator` ve `SkipListIterator`. Somut gezginler kendi veri yapılarıyla ilişkidedir.

`Client` yalnız soyut liste ve soyut gezgin üzerinden çalışır. Kendisine hangi veri yapısı ve gezgin verilirse onun içinde dolaşabilir. Böylece esneklik artar. Hoca, arka planda belirli bir veri yapısı ile belirli bir dolaşım türünün kendi aralarında bağlaşımlı olacağını, bundan kaçmanın mümkün olmadığını söyledi. Ama en azından istemci, veri yapısının ve dolaşım biçimlerinin ayrıntılarından mümkün olduğunca yalıtılmış olur. Java'nın `java.util` paketindeki veri yapıları da bu fikri daha basit bir biçimde kullanır.

> **Ek bilgi (derste anlatılmadı):** `AbstractList`'teki `CreateIterator()` metodu, hangi somut gezginin üretileceğine alt sınıfın karar verdiği bir fabrika metodudur. GoF da Iterator kalıbını Factory Method kalıbıyla birlikte kullanılan kalıplar arasında sayar.

3. Iterator Gerçekleme Ayrıntıları

Slayttaki gerçekleme ayrıntıları (ekrandan, 00:14:00) iki karar sorusu içerir.

3.1 Dolaşırken Değiştirmeye İzin Verilecek mi?

Gezgin bileşenler arasında dolaşırken bileşenlerin durum bilgisinin değiştirilmesine izin verilmeli mi? İzin verilmeyecekse veri yapısı, gezgini kendisiyle değil, kendisinin bir klonuyla ilişkilendirir. İzin verilecekse veri yapısı doğrudan kendisini gezgine verebilir.

3.2 Dolaşımı Kim Yapacak, Elemanlar Nasıl Verilecek?

Dolaşım algoritmasını kim gerçekleyecek ve dolaşılacak nesneler istekçiye birer birer mi, hepsi birden mi verilecek? Bu iki biçimde gerçeklenebilir:

- Gezgine komut verme sırasını istemci (istekçi) belirler ve parçaları gezginden tek tek okur. Örneğin her `next` çağrısında bir eleman gelir.
- İstekçi gezgine bir dolaşım isteği verir ("şu yapıyı dolaşacağım"), gezgin de bütün parçaları bir dizide döndürür.

Elemanların sırası önemliyse bu, gezginin ek metotlarıyla sağlanabilir. Yapı anahtar-değer çiftleriyle saklanan bir tabloysa elemanlar anahtarların artan ya da azalan sırasıyla döndürülebilir. Yapı bir ağaçsa LDR ya da DLR gibi dolaşım biçimleri için uygun metotlar olabilir. Farklı dolaşım biçimlerini Strategy kalıbıyla gerçeklemek de mümkündür. Hoca, Strategy'yi sevdiği bir kalıp olarak andı ve ilerideki derslerde görüleceğini söyledi.

4. Iterator Uygulama Örneği: Online Yemek Siparişi

4.1 Çözülecek Problem

Örnek Head First Design Patterns kitabından alınmıştır. Slayttaki problem şudur (ekrandan, 00:17:00):

- Online yemek siparişi verilebilecek bir firma hızla kurulmak isteniyor.
- Bu amaçla biri kahvaltılıkta, biri yemekte uzman iki firma satın alınıyor.
- Menü elemanları gerçeklemesi tesadüfen aynı denk gelmiş, ama her firmanın kendi ayrı yemek menüsü gerçeklemesi var.
- Farklı menü gerçeklemelerinden yola çıkarak birleştirilmiş menü müşterilere ortak bir kod üzerinden nasıl sunulacak?

- Ortak kod olmazsa, her yeni satın alınan firma için menü sunma koduna yeni bir kod bloğu eklemek gerekecek. Genişleme planlandığı için bu önemlidir.
- İşler daha da çetrefil hâle gelebilir. İki firma da vejetaryenleri düşünüp hangi yemeklerin et içermediğini işaretlemiş. Bir vejetaryen menü sunmak için iki firmanın menüsünü ayrı ayrı dolaşıp et içermeyenleri göstermek gerekecek.

Hoca seçeneklerin artabileceğini de ekledi: vegan, koşer, helal sertifikalı ya da glutensiz yemekler gibi. İleride bunları da yapabilmek istiyoruz. Önce ortak kod olmadan yapılan acemi çözüm, sonra Iterator kalıbıyla çözüm incelendi. Acemi çözümün kodu ders notlarında yok, ama paylaşılan kaynak kodlarda bulunuyor.

4.2 Mevcut Altyapı

Hoca Eclipse'te iki paketi gösterdi: `dp.iterator.exampleWithoutPattern` ve `dp.iterator.exampleWithPatt` (ekrandan, 00:30:00).

`MenuItem` sınıfı iki firmada ortaktır ve gayet basittir: ad, tanım (açıklama), vejetaryen olup olmadığı ve fiyat olmak üzere dört üye alanı vardır. Kurucu dördünü de alır ve her alan için `get` metotları vardır. Menü yazdırmayı kolaylaştırmak için `toString` metodu da eklenmiştir.

Yemek firmasının `DinerMenu` sınıfı arka planda dizi kullanır (ekrandan, 00:20:00, 00:30:00):

```
public class DinerMenu implements Menu {
    static final int MAX_ITEMS = 6; int numberOfItems = 0; MenuItem[] menuItems;
    public DinerMenu() {
        menuItems = new MenuItem[MAX_ITEMS];
        addItem("Vegetarian BLT",
                "(Fakin') Bacon with lettuce&tomato on whole wheat", true, 2.99);
        addItem("Regular BLT",
                "Bacon with lettuce & tomato on whole wheat", false, 2.99);
        //add more items to menu
    }
    public String toString() { return "Diner Menu"; }
    public void addItem(String name, String description,
                        boolean vegetarian, double price) {
        MenuItem menuItem = new MenuItem(name, description, vegetarian, price);
        if (numberOfItems >= MAX_ITEMS) {
            System.err.println("Sorry, menu is full! Can't add item");
        } else { menuItems[numberOfItems] = menuItem; numberOfItems++; }
    }
    public MenuItem[] getMenuItems() { return menuItems; }
    /*Iterator kalıbını kullanan iyileştirilmiş tasarımdan gelen ek*/
    public Iterator<MenuItem> createIterator() {
        return new DinerMenuIterator(menuItems);
    }
}
```

Slayttaki bu kod kalıplı sürümdür. Acemi sürümde `implements Menu` ve `createIterator()` yoktur; kurucu diziyi ilklendirip menüyü oluşturur, `getMenuItems()` diziyi doğrudan döndürür. Hoca, kapasite artırımı gibi ayrıntıların düşünülmediğini, bunlara gerek de olmadığını söyledi.

Kahvaltılık firmasının `PancakeHouseMenu` sınıfı ise generic bir `ArrayList<MenuItem>` kullanır. Kurucuda menü elemanlarını oluşturup listeye ekler, `toString` adını söyler, eleman ekleme metodu parametrelerden bir `MenuItem` oluşturup listeye ekler. `getMenuItems()` listeyi tek adımda döndürür.

4.3 Acemi Çözüm: WaitressQuickAndDirty

Acemi garson (`WaitressQuickAndDirty`) her iki menüyü de kendi somut tiplerinde üye alan olarak tutmak ve kurucusunda ilklendirmek zorundadır. Menüyü şöyle sunar: önce "MENU" ve "BREAKFAST" başlığını yazar, kahvaltılık firmasının `getMenuItems()` ile döndürdüğü `ArrayList`'teki elemanları tek tek alıp yazdırır. Sonra öğle yemeği başlığını atıp bu kez yemek firmasının dizisindeki elemanları tek tek yazdırır. Bir menü elemanının yazdırılması ayrı bir

metotta toplanmıştır: adı, virgül, fiyatı, sonra çizgi ve açıklaması. Hoca bu metodun `public` olmasına gerek olmadığını söyledi.

Vejetaryen menü de neredeyse aynıdır; yalnız her eleman için "vejetaryense yazdır" diye fazladan bir `if` vardır. Yalnız kahvaltı ya da yalnız öğle yemeği isteyen müşteri için ayrı metotlar vardır. Hoca burada bir eleştiri daha yaptı: madem kahvaltı ve öğle menüsünü basan ayrı metotlar var, genel `printMenu` içindeki kod yerine bu metotlar çağrılabilirdi. Bu hâliyle yinelenen kod oluşmuştur.

4.4 Kalıpla Çözüm

Kalıpla çözümde iki koleksiyon, `DinerMenu` ve `PancakeHouseMenu`, ortak bir `Menu` arayüzü altında toplanır ve bu arayüze bir gezgin oluşturma metodu eklenir (ekrandan, 00:20:00):

```
public interface Menu {
    public java.util.Iterator<MenuItem> createIterator();
}
```

Hoca, arayüzde metot gövdesi olamayacağı için ayrıca `abstract` yazmaya gerek olmadığını hatırlattı. Gezgin tarafında `DinerMenuIterator` ve `PancakeHouseMenuIterator` sınıfları vardır; birincisi `DinerMenu` ile, ikincisi `PancakeHouseMenu` ile ilişkidedir.

Arka plandaki menü kodu mümkün olduğunca az değiştirildi. Hoca, amacın bu aşamada refactoring yapıp kodu iyileştirmek değil, iki altyapıyı bütünleştirmek olduğunu söyledi. Menü sınıflarına yalnız `Menu` arayüzünün gereği olan `createIterator()` eklendi.

`DinerMenuIterator`, `DinerMenu`'nün dizisine bağımlıdır; kurucuda diziyi alır ve dizide dolaşma metotlarını kodlar:

- `next()`: şu anki elemanı alıp saklar, konumu bir artırır ve elemanı döndürür. Hoca gezginin elemanın kendisini döndürdüğüne dikkat çekti.
- `hasNext()`: konum dizinin eleman sayısını aşmışsa ya da o konumdaki eleman `null` ise `false` döndürür. Dizi 6 elemanlık düşünülmüş ama belki 5 eleman eklenmiştir; döngünün boş hücrede patlamaması için bu denetim konmuştur.
- `remove()`: o anki konumdaki elemanı siler (ekrandan, 00:40:00). Hoca biraz tehlikeli bir metot olduğunu, dizilerle çalışmanın listelere göre ne kadar zahmetli olduğunu göstermek için yazıldığını söyledi.

```
public void remove() {
    if (position <= 0) {
        throw new IllegalStateException
            ("You can't remove an item until you've done at least one next()");
    }
    if (list[position-1] != null) {
        for (int i = position-1; i < (list.length-1); i++) {
            list[i] = list[i+1];
        }
        list[list.length-1] = null;
    }
}
```

Hoca önce bu denetimin negatif konuma karşı olduğunu düşündü, sonra düzeltti: gezginlerde ilk `next` çağrılmadan konum ayarlanmaz, bu yüzden en az bir `next` yapılmadan eleman silinemez. Konum eleman sayısından büyükse ayrı bir denetim yazılmamıştır; hoca gerekirse iki satırla eklenebileceğini söyledi.

`PancakeHouseMenuIterator` kurucuda çalışacağı `ArrayList`'i alır. `next()` önce elemanı alır, sonra konumu bir artırır. `hasNext()` listenin uzunluğunu `size()` ile öğrenir: konum ondan büyük ya da eşitse başka eleman yoktur. `remove()` işi basitçe listenin `remove` metoduna iletir.

4.5 Yeni Garson: Waitress

Asıl kazanç altyapıda değil, garsondadır: garsona farklı menüler arasında ortak, eş biçimli bir dolaşım seçeneği sağlanmıştır. Böylece üçüncü bir yemek firması satın alındığında garsona

ekstra kod yazmak gerekmez.

`Waitress` önce kahvaltılık menüsünün gezginini alır, "MENU" ve "BREAKFAST" satırlarını yazar ve bu gezgini `printMenu(Iterator)` metoduna gönderir. Öğle yemeği için de aynısını yapar. `printMenu(Iterator)` gayet basittir: gezginin işleyeceği sıradaki kayıt olduğu sürece `next()` ile elemanı al ve yazdır. Eleman yazdırma bir önceki sürümle aynıdır. Hoca, metotları mümkün olduğunca odaklanmış parçalara bölmenin nesneye dayalı yaklaşımın önerisi olduğunu söyledi.

Vejetaryen menü için kahvaltılıktan alınan gezgin vejetaryen menü yazdıran `private` metoda, ardından öğle yemeğinden alınan gezgin aynı metoda gönderilir. Bu metot da eleman oldukça ilerler, yalnız vejetaryen olanları yazdırır. Yalnız kahvaltı menüsü isteyen müşteri için `PancakeHouseMenu`'den gezgin alınıp yine aynı `printMenu` kullanılır.

Ana programda garson ilklendirilip "menüyü bas" denir. Hoca iki çözümü çalıştırıp çıktıları bir metin düzenleyicide yan yana koydu: ikisi tamamen aynı çıktıyı verir. Önemli olan altyapıdaki iyileşmedir.

4.6 İki Çözümün Karşılaştırılması

Hoca iki garson sınıfını Eclipse'in "Compare with each other" özelliğiyle karşılaştırdı (ekrandan, 00:50:00):

```
// Acemi çözüm (exampleWithoutPattern)
public class WaitressQuickAndDirty {
    private PancakeHouseMenu pancakeHouseMenu;
    private DinerMenu dinerMenu;
    public WaitressQuickAndDirty() {
        pancakeHouseMenu = new PancakeHouseMenu();
        dinerMenu = new DinerMenu();
    }
    ...
}

// Kalıpla çözüm (exampleWithPattern)
public class Waitress {
    private Menu pancakeHouseMenu, dinerMenu;
    public Waitress(Menu pancakeHouseMenu, Menu dinerMenu) {
        this.pancakeHouseMenu = pancakeHouseMenu;
        this.dinerMenu = dinerMenu;
    }
    public void printMenu() {
        Iterator<MenuItem> pancakeIterator = ...
        ...
    }
}
```

Karşılaştırmadan çıkan farklar şunlardır:

- Eski hâlde ayrı ayrı somut menü tiplerinde üye alanlar gerekiyordu. Yeni hâlde hepsi tek bir `Menu` tipi altında toplandı. Hocaya göre en büyük kazanç budur. Bir tatlıcı firması satın alınsaydı eski çözümde `private DessertMenu` gibi üçüncü bir tipte alan gerekirdi; yeni çözümde aynı tipten üçüncü bir alan eklenir.
- Eski garsonun kurucusu menüleri kendisi oluşturuyordu; yenisi menüleri parametre olarak alıyor. Hoca bunun kalıbın gereği olmadığını, yalnız bir alternatif göstermek için yapıldığını söyledi.
- Eski `printMenu` kahvaltı ve öğle için ayrı döngüler ve yinelenen kod içeriyordu. Yeni çözüm eş biçimlidir ve en az yineleme içerir: aynı metoda hem kahvaltılığın hem yemekçinin gezgini verilebilir. Acemi çözümde bu mümkün değildir, çünkü biri listeyi, biri diziyi dolaşır.
- Üçüncü bir menü için eski çözümde ayrı bir `for` döngüsü yazmak gerekirdi; yenisinde `printMenu(dessertIterator)` gibi tek bir satır yeter.

- Eski `printMenu` tek ve uzun bir metottu. Yenisi bir `public`, bir `private` metoda ayrıldı; her biri kendi işine odaklandı. Hoca bunun, refactoring konusunda görülecek Extract Method'a bir örnek olduğunu söyledi.

Somut gezginlerin ayrıntıları elbette farklı kodlanmıştır; bu kaçınılmazdır. Ama o yapı garsonu etkilemez. Kalıbın getirdiği esneklik budur ve üçüncü, dördüncü, beşinci alternatiflerin eklenmesi kolaylaşır.

5. Memento Kalıbı: Amaç, Yapı ve Etkileşim

5.1 Ad ve Amaç

Aradan sonra Memento kalıbına geçildi. Hoca aynı adlı filmi önerdi: yakın dönem hafızasını kaybeden bir kişi, çektiği fotoğraflar üzerinden bu eksikliği gidermeye çalışır. "Memento" anı, hatıra anlamına gelir; kalıpta da nesnenin bir anının "fotoğrafı" çekilir.

Slayttaki amaç şudur (ekrandan, 01:08:45):

- Bir nesnenin durum bilgisini sarmalama (encapsulation) ilkesini bozmadan elde etmek.
- Böylece nesnelerin ikincil saklama ortamlarında saklanması, geri yüklenmesi, ağ üzerinden aktarımı mümkün olabilir.
- Yine bu şekilde son yapılan işlemi geri alma düzeneği (undo) kurulabilir. Durum bilgisinin farklı anlarda mementosu alınırsa, kaç adım çekildiyse o kadar adım geri dönülebilir.
- Durum bilgisinin yalnız bir kısmının saklanması da yeterli olabilir.

5.2 Kalıp Yapısı

Hoca, sarmalamayı bozmadan yapılabilecek çok fazla şey olmadığı için kalıbın alternatif çözümleri olmadığını söyledi. Yapı şöyledir (ekrandan, 01:10:00):

```
Originator                      Memento  <>------ Caretaker
------------------  - - - ->   -----------  memento
SetMemento(Memento m)          GetState()
CreateMemento()                SetState()
------------------             -----------
state                          state

CreateMemento():  return new Memento(state)
SetMemento(m):    state = m->GetState()
```

- `Originator`: durum bilgisini saklamak istediğimiz nesnedir. `state` alanı elbette `private`'tır ve her türlü yapıda olabilir.
- `CreateMemento()`: çağrıldığı anda durum bilgisini simgeleyen bir memento nesnesi oluşturur, durum bilgisini mementoya verir ve oluşan memento örneğini döndürür.
- `SetMemento(Memento m)`: originator'a bir memento verildiğinde, mementodaki durum bilgisini öğrenip kendi durumunu o hâle getirir.
- `Memento`: durum için `get` ve `set` metotları vardır. İstenirse yalnız kısmi durum (partial state) saklanabilir, yani bütün üye alanlar yerine bir kısmı.
- `Caretaker`: mementoları bir dizi, liste ya da tabloda tutar. İstediği anda durumun bir kopyasını alır ve gerektiğinde elindeki kopyalardan birini geçerli durum hâline getirir.

5.3 Bileşenlerin Etkileşimi

Normal çalışma biçimi slayttaki sıralama şemasında görülür (ekrandan, 01:13:00):

```
aCaretaker          anOriginator           aMemento
    |--CreateMemento()-->|                      |
    |                    |- - new Memento - - ->|
    |                    |------SetState()----->|
    |                    |                      |
   ...       (zaman geçer, originator'ın durumu değişir)
    |                    |                      |
```

```
|-SetMemento(aMemento)->|                    |
|                       |------GetState()----->|
```

`Caretaker` herhangi bir anda ilişkide olduğu `Originator`'a `CreateMemento()` der. `Originator` bir memento oluşturur, "benim durumum bu" diyerek `SetState()` ile durumu ona yazar ve işler bitince `CreateMemento()` döner. Aradan zaman geçer, originator'ın durumu değişir. Geri dönmek gerektiğinde caretaker, `SetMemento()` ile "bu, bana daha önce verdiğin memento; bu hâline geri dön" der. Originator, `GetState()` ile mementodaki değerleri alır, kendi `set` metotlarını çağırır ve denetimi caretaker'a geri bırakır.

> **Ek bilgi (derste anlatılmadı):** UML 2 sıralama şemalarında nesne oluşturma mesajı (`new Memento` gibi) genellikle kesikli çizgi ve açık okla, oluşturulan nesnenin başlığına doğru çizilir. Senkron çağrılar düz çizgi ve dolu okla, dönüş mesajları kesikli çizgiyle gösterilir.

5.4 Ne Zaman Kullanılır?

Kalıp, nesnenin durum bilgisinin tümünün ya da bir kısmının daha sonra o duruma dönebilmek üzere saklanması gerektiğinde kullanılır. Hoca burada önemli bir ayrım yaptı: nesnenin zaman içinde geçirdiği hâlleri bir yapıda tutup istediğimiz zaman geri dönmek için kalıba gerek yoktur. Kalıbın getirdiği şey, nesneye doğrudan bir işaretçi vermenin veri gizliliği ilkesini bozmasını istemediğimiz durumlar içindir. Bu sıkı denetim istendiğinde kalıp kullanılır.

6. Memento'yu Korumak: Dile Göre Çözümler

6.1 Korunması Gerekenler

Kalıp kâğıt üstünde güzeldir, ama pratikte dikkat edilmesi gerekenler vardır:

- Originator'a sahte bir memento verilememelidir. Yoksa dışarıdan biri kendi oluşturduğu bir mementoyu originator'a verip onu istediği biçime sokabilir. Yalnız o originator tarafından üretilmiş mementolar işlenebilmelidir.
- Kimse mementoyu bozamamalı, değiştirememeli, geçersiz kılamamalıdır (tamper, invalidate).

Bunun için mementonun metotlarına yalnız ve yalnız originator erişebilmelidir. Caretaker bile mementonun metotlarına ulaşamamalıdır; ulaşabilirse mementoyu bozabilir ya da sahte bir hâle getirebilir. Hoca bunun nasıl yapılacağının çok önemli bir problem olduğunu ve her dilde farklı ayrıntılar içerdiğini söyledi.

6.2 C++ Çözümü

C++'ta mementonun metotları `private` yapılır ve `friend` kuralı kullanılır: `Originator` sınıfı `Memento` sınıfının arkadaşı (`friend`) olarak tanımlanır. Böylece `Originator` bu metotlara `public`'miş gibi erişebilir, ama başka hiçbir sınıftan hiçbir nesne erişemez. Hoca bunun dilde hazır olan gayet rahat bir çözüm olduğunu söyledi.

6.3 Java: Paket Düzeyi Çözüm

Java'da işler biraz farklıdır ve iki çözüm vardır. Birincisi daha az denetim sağlar: mementonun metotlarına `public`, `private` ya da `protected` yazılmaz. Böyle bir metot paket (package) düzeyinde olur: paketteki diğer sınıflara `public`, başka paketlerdeki sınıflara `private` gibi davranır. `Memento` ile `Originator` aynı pakete konur; bu pakete konmayan sınıflar için memento erişilemez olur. Caretaker'ın mementoyu değiştirememesi isteniyorsa başka pakete, değiştirebilmesi isteniyorsa aynı pakete alınır.

6.4 Java: İç Sınıf Çözümü

C++'taki `friend` çözümüne daha yakın bir denetim sağlayan ikinci yol iç sınıftır (inner class). Bir sınıfın gövdesi içinde tanımlanan sınıfa iç sınıf denir; `public` ya da `private` olmasının farklı sonuçları vardır.

- `Memento`, `Originator`'ın `public` iç sınıfı yapılır. Çünkü caretaker memento nesnelerini alabilmeli, görebilmeli, kendinde bir kopyasını saklayabilmelidir; yani o sınıftan nesne sahibi olabilmelidir.
- Mementonun metotları `private` yapılır. Böylece caretaker mementoyu tutabilir ama değiştiremez.
- Durum sınıfı klonlanabilir (`Cloneable`) yapılır ve `createMemento()` durumun klonunu döndürmek zorundadır. Aksi hâlde problem yaşanır.

Hoca bu çözümün işe yaradığını gösterebildiğini, ama nedenini yüzde yüz açıklayamayacağını dürüstçe söyledi; açıklayabilecek biri olursa sevineceğini ekledi.

7. Memento Kod Gösterimi ve Kırma Denemeleri

7.1 Paket Düzeyi Sürüm

Eclipse'teki paketler şunlardır (ekrandan, 01:30:00): `dp.memento` (`Memento`, `Originator`, `State`), `dp.mementoOuter` (`Caretaker`), `dp.mementoV2.innerClass` (`Caretaker`, `Originator`, `State`) ve `dp.mementoV2Outer` (`Caretaker`).

Paket düzeyi sürümde `Originator` ve `Memento` ayrı, `public` sınıflardır; ama `Memento`'nun metotları paket düzeyindedir, yani `dp.memento` paketi dışındaki sınıflardan erişilemez. `Originator`'da `State` bir üye alandır ve varsayılan kurucuda ilklendirilir. `createMemento()` yeni bir memento oluşturup durumu ona yazar ve döndürür; `setMemento()` mementonun durumunu kendi durumu olarak atar. Durumun değişebildiğini göstermek için bir `modify` metodu konmuştur. `State` sınıfında yalnız tek bir `String durum` alanı vardır; `toString` bu metni ekrana yazar.

Dış paketteki `Caretaker` şunu yapar: bir `Originator` (subject) oluşturur, durumunu "ilk durum" yapar, bir memento alıp saklar, durumu "yeni durum" yapar, sonra eski mementoyu vererek eski hâle döndürür ve durumu yazdırır. Ekrana "ilk durum" yazılır. Geri dönülemeseydi "yeni durum" yazardı. Hoca, `dp.mementoV2Outer` paketindeki caretaker'ın da iç sınıf çözümünün dışarıdan kullanılabildiğini göstermek için yazıldığını, kodun aynı olduğunu söyledi.

7.2 İç Sınıf Sürümü

En fazla kısıtlamaya sahip çözüm iç sınıf sürümüdür. Burada ayrı bir `Memento` sınıfı yoktur; `Memento`, `Originator`'ın gövdesi içinde tanımlanmış `public` bir iç sınıftır. `getState` ve `setState` metotları basittir; tek özellikleri `private` olmalarıdır.

İki sürümün `State` sınıfı karşılaştırıldığında farklar şunlardır (ekrandan, 01:30:00): ikinci sürümde `setDurum` paket düzeyindedir ve sınıf `Cloneable`'ı gerçekler; ayrıca kendi `clone()` metodu vardır:

```
void setDurum(String durum) {
    this.durum = durum;
}

public State clone() {
    State s = new State();
    s.durum = new String(durum);
    return s;
}
```

`Cloneable` aslında hiçbir metodun gerçeklenmesini talep etmeyen bir işaretleyici arayüzdür (marker interface); hoca bunu Eclipse'te tip hiyerarşisini açarak doğruladı. `clone()` zaten `Object` sınıfında vardır. Yazılan `clone()` kendine bir işaretçi döndürmez; aynı durum bilgisine sahip bir kopya oluşturup değerini atar ve onu döndürür.

7.3 Kırma Denemeleri

Hoca önce iç sınıf sürümünün düzgün çalıştığını gösterdi: yine "ilk durum" yazıldı. Sonra mementoyu kırmaya çalıştı. Aklına gelen yol şuydu: bir subject oluşturup "ilk durum" vermek, durumunu değiştirmek, sonra kendi kafasından bir "fake subject" oluşturup durumunu "sahte

durum” yapmak, bu sahte subject’in mementosunu asıl subject’e verip yazdırmak. Çıktı şöyle oldu (ekrandan, 01:40:00):

```
Kırma denemesi:
sahte durum
```

Hocaya göre tek başarılı kırma budur: kendi türünden başka bir nesnenin durumunu alıp kullanmak. Bunun dışında, örneğin `main` ya da test metodu içinde alınmış bir mementonun durumunu değiştirecek hiçbir metoda erişilemez; memento nesnesi değiştirilemez. Hoca, başka bir originator’dan alınan mementonun iç bütünlüğü bozulmadan verildiğini, bu yüzden bunun geçerli bir işlem olarak da düşünülebileceğini söyledi ve yorumu dinleyicilere bıraktı.

Hoca bir de `Caretaker` ile `Originator` aynı pakette olduğu hâlde mementonun durumuna erişilemediğini gösterdi: metotlar `private` olduğu için. `private` silinip paket düzeyinde bırakılırsa aynı paketteki caretaker mementonun durumunu değiştirebilir. Bu istenmediği için metotlar yeniden `private` yapıldı.

7.4 Sahte Mementoyu Engelleme Önerileri

Başka bir originator’dan alınan mementonun başka bir originator’a verilmesini engellemek için hoca şu yolu önerdi:

- `Originator`, kurucu çağrıldığında rastgele bir `long` sayı üretip kendine kimlik olarak atar.
- `createMemento()` bu kimliği de mementonun durumuna yazar.
- `setMemento()`, mementonun durumunu atamadan önce mementonun iddia ettiği kimlikle kendi kimliğini karşılaştırır. Uyuşmuyorsa atamayı yapmaz, uyuşuyorsa yapar.

Hoca bunun bu örnek için gerekmediğini, ama nasıl yapılacağının bütün ayrıntılarını söylediğini, geriye yalnız koda dökmek kaldığını söyledi. Ders sonuna doğru başka seçenekler de ekledi:

- Kimlik olarak rastgele sayı yerine nesnenin hash kodu da kullanılabilir. İkisi de yalnız program çalıştığı sürece geçerli bir çözümdür.
- Kurucuda verilen bir seri numarası daha kalıcı bir kimlik olur.
- Mementoların saklanması caretaker’a değil originator’ın kendisine verilirse sorun tamamen çözülür. “Şu numaralı geçmiş durumuna dön” gibi işler yapılacaksa bu bir alternatif olabilir.

7.5 Klonlamanın Zorunluluğu

Hoca, durumun klonlanabilir olması ve `createMemento()`’nun klon döndürmesi gerektiğini de gösterdi. `State`’ten `implements Cloneable`’ı kaldırdı ve `clone()` metodunu yeni nesne yerine `return this` döndürecek biçimde değiştirdi. Sonuç olarak caretaker sonunda “ilk durum” yerine “yeni durum” yazdı; geri dönüş mümkün olmadı.

Sebep, Java’da nesnelerin referans değeriyle aktarılmasıdır (hoca bunu “call by value of reference” diye andı). `return this` ile memento ve originator tek bir durum nesnesini paylaşır; originator’daki `modify` işlemi mementodaki “eski” durumu da değiştirir. Klon döndürülüp `implements Cloneable` geri yazılınca geri dönüş yeniden çalıştı. Hoca, `implements Cloneable` yazılmazsa başka bir yerde göremediği bir yan etki olabileceğini düşündüğünü, bu yüzden Java’nın tavsiyesine uyup yazılmasını önerdiğini söyledi.

> **Ders sonrası not:** `Cloneable` denetimi yalnız `Object.clone()` (yani `super.clone()`) çağrıldığında yapılır; arayüz yoksa bu çağrı `CloneNotSupportedException` fırlatır. Yukarıdaki gibi `new` ile kopya üreten kendi `clone()` metodu için `Cloneable` büyük olasılıkla zorunlu değildir. Deneyde geri dönüşü bozan asıl değişiklik `return this`’tir; arayüzü yazmak ise zarar vermez.

8. Memento Uygulama Örneği: Geri Alma ve Yineleme

Slayttaki problem bir belge işleme programında geri alma (undo) ve yineleme (redo) işlemlerinin gerçeklenmesidir (ekrandan, 01:48:45).

```
Controller 1 ----- * Document
(DocumentMemento ile          - currentState : DocumentMemento
 hiçbir şekilde ilişkisi yok) - states : List<DocumentMemento>
                              - position : int
                              + undo()
                              + redo()
                              + anyOperation()
                                   1
                                   |
                                   * DocumentMemento
                                     - documentState
```

Belgenin durumlarını saklamak için `DocumentMemento` sınıfı oluşturulur; `documentState` alanı durum hangi türdense o türdendir. Bir `Document`'ın birden çok `DocumentMemento`'su olabilir. Hangisinin geçerli durum olduğunu `currentState` tutar. Geçmiş durumlar `states` listesinde, şu anki konum `position`'da saklanır. Slayttaki kod şudur (ekrandan, 01:48:45):

```
public void undo() {
    position -= 1;
    currentState = states.get(position);
}
public void redo() {
    position += 1;
    currentState = states.get(position);
}
public void anyOperation() {
    states.add(currentState);
}
```

`undo()` konumu bir azaltır ve bir önceki durumu geçerli yapar; `redo()` konumu bir artırır. Durum bilgisini değiştiren her işlem, değiştirdiği durumu listeye ekler. Hoca konumun negatife düşebileceğini ya da var olandan daha ileri bir konuma geçilebileceğini, bunların denetlenerek kurgunun sağlamlaştırılmasının daha iyi olacağını söyledi. Slayttaki açıklama notlarında da "position negatife düşebilir, engelle" uyarısı ve bunun biraz yarım bir Memento uygulaması olduğu, çünkü caretaker ile belgenin aynı sınıf olduğu notu yer alıyor (ekrandan, 01:48:45).

Hoca bu örneği iki satırlık basit bir gösterim olarak geçti. Ona göre iç sınıflarla mementonun nasıl kodlandığını, nelere müdahale edilebildiğini ve edilemediğini incelemek daha önemliydi. Memento'ya bir ders saati kadar zaman ayrılmıştı.

9. Observer Kalıbı: Amaç ve Yapı

9.1 Amaç ve Örnek

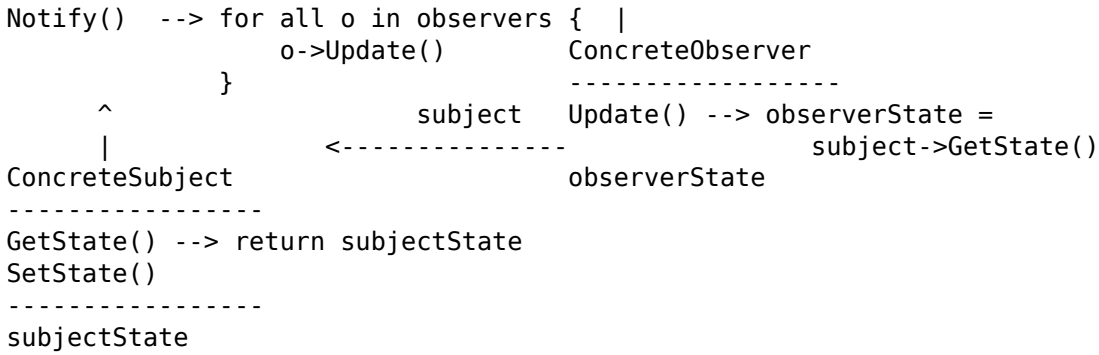
İkinci aradan sonra Observer kalıbına geçildi. Kalıbın amacı, bir nesnenin durumu değiştiğinde onunla ilişkili bütün diğer nesnelerin bu değişiklikten haberdar olmasını ve güncellenmesini sağlamaktır. Bu nesneler arasındaki ilişkiler çalışma anında kurulabilmelidir. Yani ilişkiler kodda sabit olmamalı, "bu ilişkiyi kur, bırak, ekle, çıkar" gibi metotlar çağrılarak çalışma zamanında değiştirilebilmelidir.

Hocanın örneği bir hesap tablosudur. Bir veri alanı grafiğe dönüştürülmüşse, hücrede yapılan değişiklikler anında bir ya da daha fazla grafiğe aktarılmalıdır: veri değişince çubuk grafik, akış grafiği gibi bağlı gösterimler de güncellenir. Durumu değişen nesneye bağlı nesnelerin haberdar olması, yeni durumu öğrenmesi ve kendilerini güncellemesi gerekir.

9.2 Kalıp Yapısı

Slayttaki yapı şöyledir (ekrandan, 02:10:00):

```
Subject                observers     Observer
-----------------  ---------------> ----------
Attach(Observer)                    Update()
Detach(Observer)                        ^
```



- Durum değişiklikleri önemli olan, gözlemlenecek varlıklar `Subject`'tir. Farklı türden subject'lerin gözlenmesi gerekebileceği için soyut bir `Subject` üst sınıfı vardır.
- Gözlemlenen nesneleri gözlemleyecek nesneler `Observer`'dır. Farklı gözleyici türleri olabileceği için soyut bir `Observer` sınıfı vardır.
- `Subject`'ten `Observer`'a bire-çok sahiplik ilişkisi vardır, çünkü bir subject bir ya da birden fazla gözleyici tarafından gözlenebilir. Genelde de bu gerekir.
- `Attach` ve `Detach`, gözleyici listesine yeni bir gözleyici eklemek ve çıkarmak, yani bu bire-çok ilişkiyi kurmak ve sürdürmek içindir.
- `Notify` kritiktir: subject'in durumu değiştiğinde tetiklenir ve listedeki bütün gözleyicilerin `Update` metodunu çağırmakla yükümlüdür.
- Bu üç davranış somut subject'lerden bağımsız ve ortak olduğu için soyut üst sınıfta somut metotlar olarak kodlanır.
- `ConcreteSubject`, durum bilgisini tutan alana ve onun `get/set` metotlarına sahiptir.
- Somut gözleyici, gözlemlediği somut subject'in farkındadır. Şemada bire-bir bir ilişki var; ama bir gözleyici birden fazla somut subject'i gözleyebilir, bu ek metotlarla kodlanabilir.

10. Observer Gerçekleme Ayrıntıları

10.1 Pull Model ve Push Model

İlk önemli karar, durum bilgisinin gözleyicilere nasıl gönderileceğidir (ekrandan, 02:12:00).

Pull modelde gözlenen varlık gözleyicileri yalnız haberdar eder: "ben değiştim" der, o kadar. Gözleyiciler ilgili durum bilgisini gözlenenden kendileri alır.

- Gözlenenin gözleyicilerle bağlantısını azaltan bir çalışma biçimidir. Subject farklı gözleyicilerin türünü bilmek zorunda değildir.
- Gözleyici durum bilgisinin tamamını almak zorunda değildir, yalnız istediği kısmını seçebilir. Subject'in durumunun bir alt kümesi değişmiş olabilir ve gözleyici bu alt kümenin de yalnız bir kısmıyla ilgileniyor olabilir.
- Dezavantajı: gözleyici, gözlenende nelerin değiştiğini kendisi çıkarmak ve analiz etmek zorunda kalır; kodu daha karmaşık olur.

Push model bunun tam tersidir: gözlenen, gözleyicilere durum değişikliği hakkında ayrıntılı bilgi gönderir. Bu nedenle tek bir basit `update` metodu yerine, belki aşırı yüklenmiş (overloaded) ya da farklı adlı birden fazla `update` metodu çağırmak zorunda kalabilir. Bu da subject'in kodunu karmaşıklaştırır. Hoca, karmaşıklığın iki modelden birinde mutlaka bulunacağını söyledi: pull modelde gözleyiciye, push modelde subject'e düşer.

Slayta göre (ekrandan, 02:12:00) aynı türden gözlenen nesnelerin tümünün gözleyicileri ortak olsun isteniyorsa, gözleyiciler subject içinde `static` bir veri yapısında tutulabilir.

10.2 Birden Fazla Nesne ve Farklı Türde Değişiklikler

Slayttaki devam maddeleri şunlardır (ekrandan, 02:20:00):

- Bir gözleyicinin birden fazla nesneyi gözlemesi istenebilir.
- Gözlenen nesnenin durumundaki farklı türden değişiklikleri farklı türden gözleyicilerin izlemesi istenebilir.

Hoca ikinci madde için Java'daki olay işleme (event handling), yani dinleyici (listener) mekanizmasını örnek verdi. Swing'de bir GUI bileşeninde farklı türden olaylar gerçekleşir. Bazıları pencere olaylarıdır: pencerenin odağı alması, kaybetmesi, simge durumuna küçültülmesi, büyütülmesi gibi. Bazıları fare olaylarıdır: farenin hareket etmesi, sol ya da sağ tıklama gibi. Bazıları klavye olaylarıdır: bir tuşa basılması, bırakılması gibi. Her tür ayrı dinleyicilerle izlenir. Hoca bunun daha fazla kod yazmayı gerektireceğini, ama yapılamayacak bir şey olmadığını söyledi.

10.3 Haber Verme Mesajının Tetiklenmesi

Üçüncü önemli karar, haber verme mesajının ne zaman, nasıl ve kim tarafından tetikleneceğidir. Normalde `update`'i gözlenen nesne tetikler: durum değişince bütün gözleyicilere "ben değiştim" der. GoF'un önerdiği çalışma biçimi budur. Slayttaki iki seçenek şöyledir (ekrandan, 02:20:00):

- Gözlenen nesne tetiklerse: gözlenenin istekçileri peş peşe ve hızlı durum değişiklikleri yapıyorsa, tek bir haber verme mesajıyla gönderilebilecek yerde birçok gözleyiciye gidecek bir sürü mesaj gönderilir. Ara değişiklikleri atlayıp son hâli aktarmak daha doğru olabilir. Fazla mesaj daha fazla işlem, gecikme ve haberleşme masrafı demektir. Hoca bunu IoT (Internet of Things) uygulamalarıyla örneklendi: bu cihazlarda haberleşme her zaman Wi-Fi ile değil çoğu zaman GSM üzerinden yapılır, bu da haberleşme masrafını önemli kılar.
- Gözleyiciler tetiklerse (periyodik olarak "değiştin mi?" diye sorarlarsa): gözlenenlerin istekçilerini kodlayan kişi tetikleme komutunu vermeyi unutabilir. Gözleyici bir değişikliği kaçırabilir. Sorgulamayı çok sık yaparsa bu kez de fazladan haberleşme masrafı doğar.

Bu yüzden ihtiyaçlar iyi analiz edilip bu işin ne sıklıkla ve kim tarafından yapılacağına iyi karar verilmelidir. Aksi hâlde gerekenden daha kötü bir çözüm üretilir.

10.4 Kalıbın Kullanılabileceği Anlar

Slayttaki kullanım koşulları şunlardır (ekrandan, 02:20:00):

- Herhangi bir soyutlamanın birbirine bağımlı birden fazla parçası olduğunda. Böylece değişiklik bağımlılığı verimli biçimde aktarılır.
- Bir nesnedeki değişikliklerin derleme anında bilinmeyen sayıda nesneyi etkilemesi gereken anlarda. Etkilenecek nesneler gözleyiciler, dinlenmesi gereken nesne subject olur.
- Bağlaşımı artırmadan bir nesnenin diğer nesneleri bir durumdan haberdar etmesi istenen anlarda. Arada doğrudan bir sahiplik ya da parça-bütün ilişkisi kurulmaz; subject ile observer yalnız ortak bir soyut sınıf ya da arayüz üzerinden ilişkide olduğu için bağlaşım azalır.

11. Observer Uygulama Örneği: Yeni Müşteri Eklenmesi

11.1 Problem ve İlk Çözüm

Hoca örneğin kaynağını hatırlamaya çalıştı ama bulamadı. Elimizde sipariş ve müşteri ilişkili bir program var. Sisteme yeni bir müşteri eklendiğinde müşteriye bir hoş geldin e-postası gönderilmesi ve müşterinin ikamet adresinin doğru olup olmadığının kontrol edilmesi isteniyor.

İlk akla gelen çözüm bu işi sabit biçimde kodlamaktır. `Customer` sınıfının bir `addCustomer` metodu müşteri eklenince bir `WelcomeLetter` nesnesi oluşturup hoş geldin e-postasını hazırlar; bir adres doğrulama nesnesi de adresin geçerli olup olmadığını doğrular. Hoca, aslında `addCustomer`'ın müşteri kaydının tutulacağı sınıfta olması gerektiğini, parametresiz bir metodun verileri nereye ekleyeceğini bilemeyeceğini, bunun basitleştirilmiş bir gösterim olduğunu söyledi.

Sonra gereksinimler değişir: e-ticaret sitesi bile olsak, müşterinin adresi fiziksel bir şubemize 10 kilometreden yakınsa, gönderilecek mektuba o şubede geçerli bir indirim kuponu da eklensin. Bu tip fikirlerin sonu yoktur; çok farklı kampanyalar tanımlanabilir. Bu yüzden şimdiden daha sağlam bir altyapı kurmak gerekir: sisteme her yeni müşteri eklendiğinde bir ya da birkaç

farklı işlem yapılabilmelidir. Demek ki müşteri eklendiğinde bu ekstra işleri yapacak nesnelerin haberdar edilmesi gerekir. Böyle düşününce akla Observer kalıbı gelir.

11.2 Observer ile Çözüm

Slayttaki ikinci çözüm şöyledir (ekrandan, 02:30:00):

```
Customer                                  «interface»
-----------------------------------       ICObserver
- myObs : Vector<ICObserver> {static} -->--------------------------
+ attach(ICObserver) {static}             + update(myCust : Customer)
+ detach(ICObserver) {static}                   ^ (gerçekler)
+ getState()                        +------------+--------------+
+ notifyObs()                       |            |              |
  (notifyObs metodu gözleyicilerin AddrVerification BrickAndMortar WelcomeLetter
   update metodunu çağırarak
   onları haberdar eder.)
```

Müşterileri gözlemleyecek ve her biri kendi farklı işini yapacak sınıflar vardır: `AddrVerification`, `BrickAndMortar` ve `WelcomeLetter`. Bunlar, müşterinin kendilerini haberdar edebileceği `ICObserver` arayüzünü gerçekler.

Hoca, arayüz adlarını büyük `I` ile başlatmanın güzel bir alışkanlık olduğunu, kendisinin bunu notları hazırladıktan sonra fark ettiği için birçok ders notunda bu biçimi görmeyeceğimizi söyledi. Bu adlandırmayla arayüz şemalarda kendini daha iyi belli eder.

`Customer` sınıfında gözleyicilerin tutulacağı bir `Vector` vardır. Gelen müşterinin bilgisinden ve farklı müşteri nesnelerinden bağımsız olarak, her müşteri için adres doğrulama, indirim kuponu hazırlama ve hoş geldin e-postası hazırlama aynı işler olacağı için bu gözleyici listesi `static` tutulmuştur. Hoca bunun zorunlu olmadığını, ihtiyaç öyleyse öyle yapılacağını söyledi.

Kalıp normalde soyut bir `Subject` sınıfı ve ondan türetilmiş `Customer` ister. Ama bu uygulamada yalnız müşterilerin durumundaki değişiklikler izlendiği için slayta göre soyut `Subject` üst sınıfına gerek kalmadan kalıp kullanılabilir. Yakın gelecekte müşteri dışındaki nesnelerin değişiklikleri de izlenecekse, kalıbın istediği gibi soyut bir `Subject` üst sınıfı yazılır.

11.3 Kod

Gözleyiciler `update` metodu çağrıldığında ne yapmaları gerekiyorsa onu yapar; slayttaki gözleyici kodları yalnız iskelettir. `update` parametresi olarak müşteri kendi nesne işaretçisini gönderir. `Customer` sınıfının kodu şöyledir (ekrandan, 02:32:00):

```
package observer.example;
import java.util.*;
public class Customer {
    private static Vector<ICObserver> myObs =
        new Vector<ICObserver>( ); //Gözleyiciler ortak olacak
    public static void attach(ICObserver o){
        myObs.addElement(o);    }
    public static void detach(ICObserver o){
        myObs.remove(o);    }
    public String getState () {
        // TODO: Burada durum bilgisini döndür
        return null;    }
    public void notifyObs () {
        for( ICObserver obs : myObs ) {
            obs.update(this);
        }
    }
}
```

Yeni gözleyici eklemek `Vector`'ün `addElement` metodunu, silmek `remove` metodunu çağırmaktan ibarettir. `notifyObs()`, `myObs`'taki bütün gözleyicilerin `update` metodunu çağırır ve parametre

olarak kendisini (`this`) verir. Yani gözleyicilere bütün durum bilgisini göndermiş olur. Hoca bunun gerekçesini şöyle açıkladı: farklı gözleyici türleri müşterinin farklı durum parçalarına ihtiyaç duysa da, ileride eklenecek yeni bir gözleyici müşterinin herhangi bir alanına ihtiyaç duyabilir. Programı sonradan değiştirmemek için push model ile çalışıp bütün durumu göndermek tercih edilmiştir.

> **Ek bilgi (derste anlatılmadı):** GoF, subject'in `Update` çağrısında kendisini parametre olarak geçirmesini, birden fazla subject'i gözleyen bir gözleyicinin hangi subject'in değiştiğini anlaması için önerir. Gözleyici gereken alanları bu referans üzerinden kendisi okuduğu için bu kullanım kaynaklarda pull modele de yakın sayılabilir.

Örnek Sorular

Soru 1 (ekrandan, 00:17:00): Online yemek siparişi için biri kahvaltılıkta (menüsü `ArrayList` ile), biri yemekte (menüsü dizi ile) uzman iki firma satın alınmıştır. Menü elemanı sınıfları aynıdır ama menüler farklı gerçeklenmiştir. Birleştirilmiş menü ve vejetaryen menü müşterilere, yeni firmalar eklendikçe değişmeyen ortak bir kodla nasıl sunulur?

> **Çözüm (derste verildi):** Iterator kalıbı kullanılır. İki menü sınıfı ortak bir `Menu` arayüzünü gerçekler; arayüzün tek metodu `createIterator()`'dır. `DinerMenu` dizi üzerinde dolaşan `DinerMenuIterator`'ı, `PancakeHouseMenu` liste üzerinde dolaşan `PancakeHouseMenuIterator`'ı döndürür. Garson menüleri `Menu` tipinde tutar ve tek bir `printMenu(Iterator)` metoduyla hepsini dolaşır; vejetaryen menü için de tek bir `private` metot yeter. Yeni bir firma için garsona yalnız yeni gezgini aynı metoda veren bir satır eklenir.

Soru 2 (derste sözlü, 01:32:53 ve 01:45:07): Hoca dinleyicileri iç sınıf çözümüyle korunan mementoyu kırmaya davet etti. `Memento`, `Originator`'ın `public` iç sınıfı ve metotları `private` iken, dışarıdaki bir sınıf memento üzerinden originator'ın durumunu nasıl bozabilir; bu nasıl engellenir?

> **Çözüm (derste verildi):** Mementonun durumu değiştirilemez, çünkü onu değiştiren metotlara dışarıdan erişilemez. Tek yol, aynı türden başka bir originator ("fake subject") oluşturup onun mementosunu asıl nesneye vermektir; çıktı "sahte durum" olur. Engellemek için originator kurucuda rastgele bir `long` kimlik (ya da hash kodu, ya da kalıcı bir seri numarası) üretip mementoya yazar ve `setMemento()` kimlik uyuşmazsa atamayı yapmaz. Diğer seçenek, mementoları caretaker yerine originator'ın kendisinin saklamasıdır.

Soru 3 (derste sözlü, 01:42:29): İç sınıf çözümünde durum sınıfının `clone()` metodu yeni bir kopya yerine `return this` döndürürse ne olur?

> **Çözüm (derste verildi):** Geri dönüş çalışmaz; caretaker sonunda "ilk durum" yerine "yeni durum" yazar. Memento ile originator aynı durum nesnesine referans tutar; originator'daki değişiklik mementodaki eski durumu da değiştirir. Bu yüzden `createMemento()` durumun gerçek bir kopyasını vermelidir.

Soru 4 (ekrandan, 01:48:45): Bir belge işleme programında geri alma ve yineleme işlemleri Memento kalıbıyla nasıl gerçeklenir?

> **Çözüm (derste verildi):** `Document` sınıfı geçerli durumu (`currentState : DocumentMemento`), geçmiş durumların listesini (`states : List<DocumentMemento>`) ve konumu (`position`) tutar. Durumu değiştiren her işlem `states.add(currentState)` yapar; `undo()` konumu bir azaltıp `states.get(position)`'ı, `redo()` konumu bir artırıp `states.get(position)`'ı geçerli durum yapar. `Controller`'ın `DocumentMemento` ile hiçbir ilişkisi yoktur. Konumun negatife ya da listenin dışına düşmesi denetlenmelidir.

Soru 5 (derste sözlü, 02:23:23; ekrandan, 02:30:00): Sisteme yeni bir müşteri eklendiğinde hoş geldin e-postası gönderilmesi, adresin doğrulanması ve adres bir şubeye 10 km'den yakınsa indirim kuponu eklenmesi isteniyor; ileride başka kampanyalar da eklenecek. Nasıl bir tasarım yapılır?

> **Çözüm (derste verildi):** Observer kalıbı kullanılır. `Customer` gözleyicileri `static Vector<ICObserver> myObs` içinde tutar, `static attach/detach` metotlarıyla listeyi

yönetir ve `notifyObs()` ile her gözleyicinin `update(this)` metodunu çağırır. `AddrVerification`, `BrickAndMortar` ve `WelcomeLetter`, `ICObserver` arayüzünü gerçekler. Yeni bir kampanya için yalnız yeni bir gözleyici sınıfı yazılıp `attach` edilir. Yalnız müşteriler izlendiği için soyut `Subject` sınıfı yazılmaz.

Soru 6 (derste sözlü, 02:33:41): Bir katılımcı sohbetten sordu: Müşteri örneğinde gözleyiciler için neden `Vector` yapısı kullanılıyor?

> **Çözüm (derste verildi):** Java koleksiyonları iş parçacığı güvenli (thread-safe) ve güvenli olmayan diye ikiye ayrılır. Farklı iş parçacıkları ortak veriyi kullanırken yarış koşulu oluşabilir; örneğin bildirim sırasında müşterinin adresi değişirse bir gözleyici eski, diğeri yeni adresi görebilir. Güvenli yapılar `synchronized` kilitleme kullandığı için daha yavaştır; `ArrayList` ve `LinkedList` daha hızlıdır ama çok iş parçacıklı uygulamada tutarsızlığa yol açar. `Vector` güvenli olduğu için seçilmiştir. Concurrent yapılar da, platform sağlıyorsa rahatça kullanılabilir.
>
> **Ders sonrası not:** Hoca `Vector`'ü "thread safe bir linked list" diye tanımladı; doğrusu, `Vector` dizi tabanlı ve metotları `synchronized` olan, büyüyebilen bir listedir. Hoca ayrıca Java'nın semafor gibi yapıları hazır olarak desteklemediğini, bunların ek kütüphanelerle geldiğini söyledi; `Semaphore` gibi yapılar büyük olasılıkla Java 5'ten beri standart `java.util.concurrent` paketinde bulunmaktadır.

Dersin Toparlaması

Bu derste davranışsal kalıplardan Iterator, Memento ve Observer işlendi. Iterator, bir yapının parçalarını iç yapısını açığa çıkarmadan dolaşmayı sağlar; ikinci çözümde istemci yalnız soyut liste ve soyut gezgin arayüzlerini bilir. Java'nın kendi `Iterator`'ı kalıbın basitleştirilmiş hâlidir ve KISS gereği yettiği yerde o kullanılır. Head First'ten alınan menü örneğinde, dizi ve liste kullanan iki farklı menü `Menu.createIterator()` arayüzüyle birleştirildi. Böylece garson kodundaki yineleme kalktı ve yeni bir firma eklemek tek satıra indi.

Memento, nesnenin durumunu sarmalamayı bozmadan dışarıda saklamayı sağlar. Kalıbın asıl zorluğu, mementoya yalnız originator'ın erişebilmesini sağlamaktır: C++'ta `friend`, Java'da paket düzeyi erişim ya da `private` metotlu `public` iç sınıf ile bu yapılır; durumun klonlanması da şarttır. Observer, bir nesnenin değişikliklerini çalışma anında kurulan bire-çok ilişkiyle bağımlı nesnelere bildirir. Pull ya da push modeli ve tetiklemeyi kimin yapacağı ihtiyaca göre seçilir. Müşteri örneği, yeni kampanyaların yalnız yeni gözleyici sınıflarıyla eklenebildiği bir altyapı gösterdi. Hoca bir sonraki kalıp olarak Strategy'ye geçmeyi düşündüğünü, ama bu derste geçmediğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.